

CrazyCine - Plataforma de Visualização e Alocação de Filmes

Integrantes: Marcos Filipe de Paula Macêdo; Tailon Cypreste Brandinni; Pedro Henrique dos Santos Amorim.

Disciplina: Desenvolvimento Web.

Entrega: Código fonte compactado e documentação técnica em PDF.

1. Descrição geral do produto

O CrazyCine é uma aplicação web desenvolvida em Flask para permitir que usuários visualizem, pesquisem, organizem, aluguem e comprem filmes. O sistema resolve a dificuldade de centralizar listas de filmes, acompanhar itens assistidos e controlar o acesso a filmes comprados ou alugados.

O usuário pode criar conta, fazer login, buscar filmes por nome, filtrar por gênero, visualizar detalhes, criar listas personalizadas, marcar filmes como assistidos, gerar pagamento Pix via Mercado Pago e liberar filmes após aprovação real ou simulação de pagamento no ambiente de testes.

2. Funcionalidades implementadas

Funcionalidade	Descrição
Cadastro e login	Usuários podem criar conta e acessar o sistema com senha criptografada.
Catálogo de filmes	Lista com 24 filmes, pôster, gênero, ano, diretor, aluguel e compra.
Busca e filtro	Busca por nome e filtro por gênero.
Listas personalizadas	Criação, remoção e visualização de listas, como Favoritos e Assistir depois.
Assistidos	Marcação e desmarcação de filmes como assistidos.
Aluguel e compra	Checkout Pix para aluguel ou compra de filmes.
Mercado Pago	Geração de QR Code Pix, copia e cola e webhook de confirmação.
Simulação de pagamento	Botão de teste para aprovar pagamento em apresentação.
Meus Filmes	Histórico de compras/aluguéis e status de liberação.

3. Arquitetura adotada

A arquitetura foi organizada com padrão Application Factory, usando uma função `create_app()` responsável por criar a aplicação, carregar configurações, inicializar extensões, registrar Blueprints e inicializar o banco. Isso melhora modularidade, testabilidade e separação de responsabilidades.

A aplicação usa Blueprints para separar rotas por responsabilidade: main, auth, movies, lists e payments. A camada de modelos fica em `app/models.py`, formulários em `app/forms.py`, extensão SQLAlchemy em `app/extensions.py` e integração de pagamento em `app/services/payments.py`.

Estrutura de pastas

```
CrazyCine_Final/
├── app/
│   ├── __init__.py           # create_app e registro dos Blueprints
│   ├── extensions.py         # SQLAlchemy
│   ├── models.py             # Entidades do banco
│   ├── forms.py              # Flask-WTF / WTForms
│   ├── seed.py               # Carga inicial de filmes
│   ├── services/payments.py  # Mercado Pago Pix
│   ├── blueprints/           # Rotas separadas
│   ├── templates/            # Jinja2
│   └── static/css/style.css  # Estilo customizado
├── tests/test_routes.py      # Testes automatizados
├── config.py                 # Configurações e variáveis de ambiente
├── run.py                    # Entrada da aplicação
├── requirements.txt
└── render.yaml
```

4. Diagrama do banco de dados

Modelo ER simplificado das entidades principais:

Tabela	Campos principais	Relacionamentos
User	id, name, email, password_hash	1:N com MovieList e Transaction
Movie	id, title, description, genre, year, director, poster_url, rent_price, buy_price	1:N com MovieList; 1:N com Transaction e WatchedMovie
MovieList	id, name, user_id	Pertence a User; N:N com Movie
list_movies	list_id, movie_id	Tabela associativa entre MovieList e Movie
WatchedMovie	id, user_id, movie_id	Registra filmes assistidos por usuário
Transaction	id, user_id, movie_id, type, price, status, mp_payment_id, pix_qr_code	Pedido de aluguel, compra e controle de liberação

5. Detalhamento técnico do código

Application Factory

```
def create_app(config_class=Config):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)
    db.init_app(app)
    app.register_blueprint(main_bp)
    app.register_blueprint(auth_bp)
    app.register_blueprint(movies_bp)
    app.register_blueprint(lists_bp)
    app.register_blueprint(payments_bp)
    with app.app_context():
        db.create_all()
        seed_movies()
    return app
```

Exemplo de modelo ORM

```
class Transaction(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    movie_id = db.Column(db.Integer, db.ForeignKey('movie.id'), nullable=False)
    type = db.Column(db.String(20), nullable=False)
    status = db.Column(db.String(30), nullable=False, default='pendente')
    mp_payment_id = db.Column(db.String(80), unique=True)
    pix_qr_code = db.Column(db.Text)
    expires_at = db.Column(db.DateTime)
```

Formulários e validação

Os formulários de cadastro, login e criação de listas usam Flask-WTF/WTFForms com validações no servidor, incluindo e-mail válido, campos obrigatórios, tamanho mínimo de senha e confirmação de senha.

Pagamentos

A integração com Mercado Pago gera pagamento Pix com QR Code e copia e cola. O webhook recebe notificações de pagamento e sincroniza o status. Em ambiente de teste, a aplicação também possui rota protegida para simular pagamento aprovado.

6. Tecnologias utilizadas

Tecnologia	Justificativa
Flask	Framework web leve e adequado ao escopo do MVP.
Blueprints	Separação das rotas por responsabilidade.
Flask-SQLAlchemy	ORM para persistência e modelagem das entidades.
SQLite	Banco simples para desenvolvimento e apresentação.

Flask-WTF / WTForms	Formulários com validação no servidor.
Bootstrap 5	Interface responsiva e rápida de desenvolver.
Mercado Pago SDK	Geração de Pix e consulta de status.
pytest	Testes automatizados de rotas e casos principais.
gunicorn	Servidor WSGI para deploy no Render.
Render	Deploy gratuito com subdomínio para apresentação.

7. Como rodar o projeto

1. Extrair o arquivo ZIP.
2. Abrir terminal na pasta do projeto.
3. Criar ambiente virtual:
python -m venv venv
4. Ativar no Windows:
venv\Scripts\activate
5. Instalar dependências:
pip install -r requirements.txt
6. Copiar variáveis de ambiente:
copy .env.example .env
7. Rodar localmente:
python run.py
8. Acessar:
http://127.0.0.1:5000

8. Como rodar os testes

```
pytest
```

Os testes cobrem página inicial, cadastro, login, proteção de rota, catálogo e simulação de pagamento aprovado.

9. Deploy no Render

```
Build Command:
pip install -r requirements.txt

Start Command:
gunicorn run:app

Environment Variables:
SECRET_KEY=uma-chave-segura
MERCADO_PAGO_ACCESS_TOKEN=seu-token
USE_DEMO_PIX_VALUE=true
ENABLE_TEST_PAYMENT_SIMULATION=true
WEBHOOK_BASE_URL=https://seu-app.onrender.com
```

Para apresentação, SQLite pode ser usado no Render. Para produção real, recomenda-se migrar para PostgreSQL, pois o armazenamento local em serviços gratuitos pode não ser persistente em todos os cenários.

10. Principais desafios e soluções

Integração Pix: o Mercado Pago exige credenciais válidas, e-mail de pagador em formato válido e URL pública para webhook. A solução foi isolar a integração em `app/services/payments.py` e usar variáveis de ambiente.

Testes com pagamento: como Pix de teste não é pago por banco real, foi criado um botão controlado por `ENABLE_TEST_PAYMENT_SIMULATION` para simular aprovação em apresentação.

Organização: o projeto inicial poderia ficar concentrado em um único `app.py`. A solução foi aplicar Application Factory, Blueprints e separação por camadas.

Deploy: foi incluído `requirements.txt` com gunicorn e `render.yaml` para facilitar implantação no Render.